ENGINEERING DELIVERY SPECIFICATION

Development, Testing & Deployment

Professional AC Core Cutting Business Website • v1.0 • Production Baseline

STAGE	OBJECTIVE	PRIMARY OUTPUT
Development	Build maintainable features from approved requirements	Working code + tests
Testing	Verify behavior, quality, security and conversion paths	Verified release candidate
Staging	Validate production-like build safely	Approved deployment candidate
Deployment	Release reliably with controlled risk	Production release
Verification	Confirm the live system works	Smoke-tested production
Maintenance	Keep system healthy and current	Stable ongoing operation

Master delivery flow: PLAN → DEVELOP → REVIEW → TEST → PREVIEW/STAGE → APPROVE → DEPLOY → SMOKE TEST → MONITOR → IMPROVE

This document defines the engineering delivery process for the project's approved architecture, public routes, enquiry flow and production requirements.

1. Development Principles

• Implement approved requirements before adding unnecessary features.

• Keep public pages fast, semantic and SEO-friendly.

• Prefer server-rendered/static public content and minimal client-side JavaScript.

• Build reusable components instead of page-specific duplication.

• Keep business content separate from presentation logic.

• Treat all browser input as untrusted and validate it on the server.

• Use version control for every code, schema and configuration change.

• Keep local, preview/staging and production environments separate.

• Make changes small enough to review, test and roll back safely.

• Do not ship placeholders, fake reviews, fake imagery or unverifiable claims.

Recommended stack baseline

Next.js + TypeScript + Tailwind CSS, reusable UI components, server-side enquiry handling, PostgreSQL-backed content/lead data, optimized responsive media, SEO configuration and production analytics.

2. Development Environment

AREA	STANDARD
Source control	Git repository with meaningful commits and reviewed changes
Runtime	Node.js environment matching the locked project configuration
Package manager	Use one project-approved package manager consistently
Environment variables	Local-only .env; production secrets supplied by deployment platform
Database	Local development database separate from staging/production
Media	Optimized local/test assets; production assets managed separately
Quality commands	Type check, lint, test and production build
Documentation	Keep setup, environment and deployment instructions current

Local setup flow

CLONE → INSTALL → CONFIGURE ENV → START DATABASE → RUN MIGRATIONS → LOAD SAFE DEV DATA → START APP → VERIFY

Development credentials and data must never be reused for production.

3. Project Development Workflow

Requirement → task → branch → implementation → self-review → automated checks → pull request/code review → preview → acceptance → merge

Branching model

BRANCH	PURPOSE
main	Production-ready code
feature/*	New feature or page
fix/*	Bug or defect correction
chore/*	Maintenance/tooling/dependency work

Commit quality

- Keep commits focused and understandable.
- Do not commit secrets, generated build output or local credentials.
- Reference the feature/bug being changed when practical.
- Avoid mixing unrelated refactors into a feature change.

4. Feature Development Process

STEP	DEVELOPER ACTION	DONE WHEN
1. Understand	Read requirement and acceptance criteria	Scope is clear
2. Design	Map UI, data, API and states	Implementation approach is defined
3. Build	Implement reusable components/services	Feature works locally
4. Validate	Run type/lint/test/build checks	Automated checks pass
5. Review	Inspect security, accessibility, responsiveness	No obvious blocker
6. Preview	Deploy candidate to preview/staging	Integrated behavior verified
7. Accept	Confirm against requirement	Feature approved
8. Merge	Merge controlled change	Main branch remains releasable

Every feature should include success, loading, empty and error states where applicable.

5. Page & Component Development

• Header/navigation: desktop and mobile states, active routes and keyboard access.

• Hero: primary value proposition plus Call/WhatsApp/Get a Quote actions.

• Service cards: reusable data-driven links to dedicated service pages.

• Trust sections: verified facts only.

• Gallery: optimized responsive images and meaningful alt text.

• Reviews: approved content with appropriate attribution.

• FAQ: accessible disclosure behavior and concise customer language.

• Contact form: explicit labels, validation, loading, success and safe error states.

• Sticky mobile actions: Call and WhatsApp remain easy to access without obstructing content.

• Footer: business details, navigation, contact paths and relevant legal/support information.

Reusable content model

Business Profile → ***Services*** → ***Service Areas*** → ***Gallery*** → ***Reviews*** → ***FAQs*** → ***Enquiries*** → ***Analytics Events***

6. Database & Backend Development

Backend work must preserve the approved relational model and server-side security boundary.

AREA	DEVELOPMENT RULE
Schema	Use version-controlled migrations
Queries	Parameterized/safe database access only
Validation	Validate request data before persistence
Relations	Respect FK constraints and allowed service/status values
Indexes	Maintain indexes for common public and operational queries
Enquiries	Server controls status and operational fields
Errors	Return safe public responses; log useful diagnostics server-side
Transactions	Use transactions where multiple dependent writes require atomicity

Recommended enquiry boundary: POST /api/enquiries. The endpoint validates, normalizes, protects against abuse, persists the lead and then triggers the configured lead-delivery workflow.

7. Automated Quality Checks

CHECK	PURPOSE	RELEASE EXPECTATION
Type checking	Catch TypeScript/type contract errors	Pass
Linting	Catch code-quality/common mistakes	Pass
Unit tests	Verify isolated logic	Pass
Integration tests	Verify API/data interactions	Pass
Build	Prove production compilation	Pass
Migration validation	Prove DB changes are reproducible	Pass
Secret scanning	Prevent credential exposure	No findings
Dependency audit	Identify known package risks	No critical blocker

Exact tools/commands can be chosen during implementation; the process requirement is that equivalent checks exist and are executed consistently.

8. Functional Testing

FLOW	TEST CASES
Navigation	Every approved route opens; internal links resolve; mobile menu works
Hero CTAs	Call, WhatsApp and quote actions use correct verified destinations
Services	Overview links to each service; detail pages show correct content
Gallery	Images load, responsive behavior works, categories behave as intended
Reviews	Approved reviews render correctly; no accidental placeholder content
FAQ	Open/close states work with keyboard and touch
Contact	Valid submission succeeds; invalid submission is blocked clearly
Form errors	Server errors show safe, understandable recovery guidance
Map	Map/directions link works when configured
Analytics	Approved conversion events fire on intended actions

9. Responsive, Accessibility & Browser Testing

Responsive

• Test phone-first layouts, then tablet and desktop.

• Check sticky actions do not cover content or form controls.

• Verify images, cards, tables and navigation adapt without horizontal overflow.

• Test touch targets and form usability on small screens.

Accessibility

• Semantic HTML and logical heading hierarchy.

• Keyboard navigation and visible focus states.

• Labels and useful error messages for form controls.

• Meaningful alt text for informative images.

• Sufficient contrast and readable text.

• Accessible mobile navigation, FAQ and interactive controls.

Browsers

Validate the production experience in current major desktop/mobile browser environments relevant to the target audience, with special attention to mobile Chrome and Safari.

10. SEO & Performance Testing

AREA	VERIFY
Metadata	Unique title/description for important public routes
Canonical	Canonical URLs are correct
Sitemap	Public intended URLs are included
Robots	Indexing directives match the production intent
Structured data	LocalBusiness/Service data uses verified information
Internal links	Services and local-intent content are discoverable
Images	Responsive, compressed, correctly sized, useful alt text
Fonts/JS	No unnecessary blocking resources
Core Web Vitals	Major layout, loading and interaction issues addressed

Do not add third-party widgets, maps or scripts until their performance and privacy cost is justified.

11. Security Testing

• Attempt invalid and oversized form inputs.

• Verify server-side validation cannot be bypassed by removing client validation.

• Test repeated enquiry submissions and rate limiting.

• Confirm database/API secrets never appear in browser bundles.

• Review response headers and error responses.

• Check authorization on any future private/admin route.

• Review dependencies for critical known vulnerabilities.

• Verify logs do not unnecessarily contain raw customer form data.

Security testing complements the separate Security, Quality & Operations document; this document defines when those controls are tested during delivery.

12. Staging / Preview Environment

Staging should behave like production while remaining isolated from real customer operations.

AREA	STAGING RULE
Domain	Separate preview/staging URL
Database	Separate staging database
Secrets	Separate credentials
Lead delivery	Use safe test destination or controlled test mode
Analytics	Separate/test property where appropriate
Content	Production-like, owner-approved test content
Migrations	Run against staging before production
Smoke tests	Complete before release approval

Never accidentally send staging test leads to the real business lead inbox.

13. Release Candidate & Approval

FEATURE COMPLETE → CI CHECKS → STAGING → QA → SECURITY REVIEW → CONTENT REVIEW → BUSINESS APPROVAL → RELEASE

APPROVER	RESPONSIBILITY
Developer	Technical correctness and automated checks
QA/Reviewer	Functional, responsive, accessibility and regression checks
Business owner	Business facts, services, areas, images and reviews
Release owner	Deployment readiness, migration and rollback plan

Release blocker examples

- Build failure or critical runtime error.
- Broken enquiry/Call/WhatsApp conversion path.
- Critical security issue or exposed secret.
- Unverified business claim or incorrect contact information.
- Failed database migration or untested destructive change.
- Major mobile layout or accessibility failure.

14. Production Deployment

APPROVED RELEASE → BACKUP/CHECK → DEPLOY → MIGRATE IF REQUIRED → VERIFY → SMOKE TEST → MONITOR

Deployment procedure

- Confirm approved release/commit.
- Confirm production environment variables and secrets.
- Confirm database backup/recovery readiness before risky schema changes.
- Deploy application.
- Apply approved database migrations in the correct order.
- Verify application health and key routes.
- Run production smoke tests.
- Verify Call, WhatsApp and enquiry paths.
- Verify analytics/conversion events where configured.
- Monitor errors and lead delivery after release.

15. Production Smoke Test

CHECK	EXPECTED RESULT
Homepage	Loads without console/server errors
Navigation	Primary routes open
Service page	Correct service content and CTA
Call	Correct verified phone destination
WhatsApp	Correct verified WhatsApp destination
Quote form	Validation and controlled test submission work
Database	Expected enquiry record created when tested
Email/CRM	Lead reaches configured destination when applicable
Mobile	Sticky actions and form usable
SEO	Metadata, canonical, sitemap/robots available

A production deployment is not complete until the live system—not only the build pipeline—passes smoke tests.

16. Rollback & Recovery

If a deployment causes a critical regression, prioritize restoring the last known-good state.

DETECT → STOP/CONTAIN → ROLLBACK APP → ASSESS DB → RESTORE/MIGRATE SAFELY → SMOKE TEST → MONITOR

- Keep a known-good production release available.
- Do not blindly roll back database schema changes that are not backward-compatible.
- Prefer additive, backward-compatible migrations where practical.
- Back up before high-risk migrations.
- After rollback, verify the enquiry path and core public routes.
- Document the failure and corrective action.

17. Continuous Maintenance

MAINTENANCE AREA	ACTION
Dependencies	Review and update on a controlled schedule
Security	Review alerts, secrets, headers and access
Performance	Review Core Web Vitals and heavy assets
Content	Verify business facts, services, areas, gallery and reviews
Database	Monitor storage, backups and migration health
Analytics	Confirm conversion events remain useful
SEO	Check broken links, indexing and metadata quality
Backups	Confirm backup success and periodically test recovery

Maintenance changes follow the same review discipline as feature work; production should never become the place where unreviewed experiments are made.

18. Deployment Checklist

Before deployment

- Approved release identified.
- Automated checks pass.
- Preview/staging accepted.
- Security checks pass.
- Content verified.
- Database migration reviewed.
- Backup/recovery plan confirmed.
- Production secrets/config confirmed.
- Rollback target identified.

After deployment

- Production homepage and core routes verified.
- Call/WhatsApp tested.
- Quote form tested.
- Lead persistence/delivery verified.
- Mobile behavior checked.
- Logs/monitoring checked.
- Analytics checked.
- No critical regression observed.

19. Definition of Done

- Requirement and acceptance criteria are satisfied.
- Code is reviewed and maintainable.
- Type/lint/unit/integration/build checks pass.
- Functional, responsive and browser testing pass.
- Accessibility checks pass for affected flows.
- SEO/performance checks pass for affected public routes.
- Security checks pass with no critical blocker.
- Database changes are version-controlled and tested.
- Preview/staging verification is complete.
- Business content is approved.
- Production smoke test passes.
- Monitoring and rollback expectations are in place.

The feature is considered complete only when it is technically correct, user-correct, business-correct and deployable.

20. Final Development → Testing → Deployment Lock

DEVELOP

PLAN → BRANCH → BUILD → REVIEW → AUTOMATE

TEST

FUNCTIONAL → RESPONSIVE → ACCESSIBILITY → SEO → PERFORMANCE → SECURITY

DEPLOY

STAGE → APPROVE → BACKUP/CHECK → RELEASE → SMOKE TEST → MONITOR

FINAL RULE

No production release should bypass the defined testing and approval gates merely because the change appears small.